

Ementa Engenharia de Software em IA Aplicada

1 - Fundamentos de IA e LLMs para Programadores

- **História e contexto da IA**
 - De onde veio a primeira ideia de “máquina inteligente” na humanidade
 - Linha do tempo até chegarmos em ML, DL, LLMs e ferramentas atuais
- **Introdução a LLMs (Large Language Models)**
 - Revisão de ML, DL e AI para preparar o terreno
 - Como funcionam LLMs: transformers, embeddings, attention
 - Exemplos de LLMs: ChatGPT, Gemini e outros
- **Fundamentos de Web Machine Learning**
 - Diferença entre Machine Learning, Deep Learning e Inteligência Artificial
 - Como tensores funcionam e por que são a base dos modelos
 - Conceito de redes neurais e como elas aprendem
- **Criando sua primeira IA do zero**
 - Passo a passo para criar e treinar a primeira rede neural em JavaScript
 - Entender o ciclo completo: dados → treino → validação → inferência
- **Projetos práticos com ML na Web**
 - Usar ML com vídeo no browser (ex: detecção, tracking, filtros, etc.)
 - Aplicar ML em jogos (ex: bots inteligentes, assistentes, efeitos dinâmicos)
 - Introdução prática a Deep Learning em cenários reais

- **Prompt Engineering na prática**
 - Como escrever prompts que realmente geram boas respostas
 - Padrões de prompts para codar, debugar, documentar e aprender mais rápido
- **Ferramentas de IA para acelerar sua vida como Dev**
 - História e visão geral do Cursor
 - Comparando VSCode, Windsurf e Cursor
 - Como usar `.cursor rules` para deixar a IA “do seu jeito”
 - Conceito de Vibe Coding e o case do levelsio (microSaaS com +100k/mês usando AI)
- **MCPs e automação para devs**
 - O que são MCPs (Model Context Protocol) e como conectam IA com APIs/serviços
 - Usando IA para:
 - Gerar testes automatizados
 - Consultar documentações atualizadas
 - Navegar em sites e extrair informações
 - Ferramentas como N8N MCP e similares para orquestrar fluxos com IA
- **RAG, embeddings e busca semântica**
 - O que é RAG (Retrieval-Augmented Generation) e por que ele é tão importante
 - O que são embeddings e Vector Databases
 - Conceito de similarity search (busca por “significado”, não só por texto exato)
 - Projeto: Criando o primeiro RAG com JavaScript e Postgres

- **Modelos open-source vs. proprietários**
 - Vantagens e desvantagens de modelos abertos e fechados
 - Como usar OpenRouter para orquestrar vários modelos
 - Como rodar modelos localmente com Ollama
- **Agents e automação avançada**
 - O que são agentes de IA e como eles tomam decisões em etapas
 - Exemplos reais de agentes autônomos que escrevem código e aceleram o trabalho do dev

2 - APIs de IA Generativa e Prompt Engineering

- **Panorama do mercado de IA como serviço**
 - Casos de empresas que são “ChatGPT wrappers” e levantaram milhões de dólares
 - Entendendo onde estão as oportunidades reais para devs e negócios
- **Principais provedores de modelos de IA**
 - Visão geral de OpenAI, Anthropic, Hugging Face e Google Gemini
 - Diferenças de APIs, modelos e cenários de uso
- **Engenharia de prompts avançada**
 - Prompt chaining (encadear prompts para tarefas complexas)
 - Prompt templates para padronizar fluxos e reutilizar prompts
 - Estratégias para reduzir alucinações e melhorar a precisão
- **Boas práticas de consistência e custo-eficiência**
 - Como desenhar chamadas de API pensando em custo
 - Técnicas para cache, reuso de contexto e redução de tokens

- Padrões para tornar o comportamento dos modelos mais previsível
- **RAG avançado na prática**
 - Uso de LangChain, Pipes (ou similares) para orquestrar fluxos de IA
 - Estratégias de gerenciamento de erros e re-tentativas
 - Boas práticas de logging, observabilidade e depuração em apps de IA
- **Mão na massa: integrando IA ao back-end**
 - Projeto: integrar um LLM da OpenAI a um back-end já existente
 - Fluxo de requisições, autenticação, versionamento e testes
- **Modelos multimodais (texto, imagem, áudio, vídeo)**
 - Quando usar modelos de texto, imagem, áudio e vídeo
 - Visão geral de OpenAI Vision, Whisper, Gemini multimodal e outros
- **Aplicações práticas com foco em APIs**
 - Construindo um OCR inteligente para extrair e entender documentos
 - Análise de mídia (imagem/vídeo) para insights e automações
 - Criação de bots multimodais que entendem texto, imagem, áudio e vídeo juntos

3 - MCP – Model Context Protocol

- **Visão geral de MCP (Model Context Protocol)**
 - O que é MCP e por que está ganhando tanta tração agora
 - Como ele resolve o caos de integrações customizadas entre LLMs e serviços
- **Integração padronizada de LLMs com MCP**
 - Como usar MCP para conectar LLMs a APIs, bancos de dados e serviços internos



- Padronização de entrada/saída e contexto entre diferentes modelos e ferramentas
- **MCPs vs Tools tradicionais**
 - Diferença entre MCPs e o modelo clássico de “tools/plugins”
 - Quando faz sentido usar MCP em vez de ferramentas proprietárias de cada provedor
- **MCP em JavaScript**
 - Como criar e consumir MCPs usando JavaScript/TypeScript
 - Boas práticas de organização de código e estrutura de projeto
- **Transformando sua empresa em um “MCP”**
 - Expor serviços internos (CRM, billing, suporte, dados de produto) via MCP
 - Desenhar uma camada de AI-ready para toda a stack da empresa
- **Segurança e governança em MCPs**
 - Autenticação e Service Tokens
 - Rate limiting para evitar abusos e custos inesperados
 - Web Application Firewall (WAF) e proteções para expor MCPs com segurança para LLMs e agentes
- **Casos de uso reais em empresas**
 - Como empresas usam MCPs para conectar IA a sistemas legados
 - Exemplos de automações, copilots internos e assistentes corporativos
- **Cenários práticos de aplicação**
 - Plugins e agentes inteligentes consumindo MCPs
 - Integrações com IDEs (copilots customizados)
 - Conexões com sistemas corporativos (ERP, CRM, suporte, analytics)

- **Hands-on: do zero ao MCP em produção**
 - Criação de um MCP server do zero
 - Integração desse MCP com front-end (UI) e back-end
 - Testando o MCP em diferentes LLM clients e compondo com outros MCPs

4 - Criação de Agentes Autônomos

Unidade 1 — Arquitetura de Agents

- Conceito de *agent loop*: percepção → raciocínio → ação → feedback.
- Estrutura interna: *planner*, *executor*, *memory store*, *toolbox*.
- Tipos de agentes: *task-based*, *interactive*, *goal-oriented*, *autonomous*.
- Componentização: prompts, ferramentas, contexto, e feedback loops.

Unidade 2 — Padrões de Raciocínio e Execução

- **ReAct Pattern**: raciocínio + ação iterativa (reasoning traces).
- **Plan-and-Execute Pattern**: decomposição de objetivos e execução estruturada.
- **Reflection Pattern**: autoavaliação e correção de comportamento.
- Comparativo prático entre ReAct e Plan-Execute em diferentes cenários.
- Implementações com LangChain, LangGraph e frameworks equivalentes.

Unidade 3 — Function Calling e Tool Use

- Princípios de *tool use* e *function calling* em LLMs.
- Design de *schemas* JSON para funções executáveis.
- Conexão de agentes a APIs REST, bancos de dados, automações e MCPs.
- Implementação prática: agente que cria, executa e ajusta tarefas com ferramentas externas.

Unidade 4 — Memória e Reflexão em Agentes Autônomos

- Tipos de memória: curta, longa, episódica e contextual.
- Estratégias de armazenamento e recuperação via embeddings.
- *Reflection loops*: como permitir que agentes revisem e melhorem sua própria lógica.
- Casos de uso: assistentes pessoais, debugging automatizado e aprendizado incremental.

Unidade 5 — Gerenciamento de Contextos

- O que é um “contexto ativo” e como estruturá-lo.
- Técnicas de *context pruning* e *context stitching*.
- Contextos compartilhados entre múltiplos agentes.
- Balanceamento entre contexto global (memória coletiva) e local (memória individual).

Unidade 6 — LangGraph e Workflows Complexos

- Introdução ao **LangGraph** como orquestrador de fluxos multiagente.
- Criação de grafos de execução e controle de dependências.
- Roteamento de tarefas, fallback handlers e retry logic.
- Monitoramento e depuração de agentes conectados em grafos.

Unidade 7 — Observabilidade e Limites de Autonomia

- Métricas e logs de execução de agentes.
- Auditoria e rastreabilidade de decisões autônomas.
- Implementação de guardrails e *human-in-the-loop*.
- Delimitação de escopo e ética na autonomia de agentes.
- Prevenção de *runaway loops* e abusos de API.

Unidade 8 — Projeto Prático: Agente que Automatiza Tarefas Complexas

- Desenvolvimento de um **agente autônomo completo**, com:
 - planejamento adaptativo;
 - execução de tarefas via APIs externas;
 - registro de logs e feedback loops;
 - controle de contexto e memória persistente.
- Integração com **LangGraph** para orquestrar subtarefas.
- Apresentação e avaliação com base em métricas de eficácia, autonomia e resiliência.

Unidade 9 — Multi-Agent Systems

- Conceitos de colaboração entre agentes: *orquestração, coordenação e negociação*.
- Padrões: *Supervisor, Hierarchical, Group Chat, Delegation e Consensus*.
- Comunicação assíncrona e troca de mensagens.
- Implementação de um **mini sistema multiagente cooperativo** (ex.: analista → planejador → executor).
- Casos reais: Claude Team, AutoGPTs, CrewAI, Agentarium (referência prática).

5 - Ferramentas de IA para UX & UI

Unidade 1: A Revolução da IA no Design e Desenvolvimento de Interfaces

- Introdução ao conceito de AI-driven UX/UI.
- Panorama das ferramentas de IA para cada etapa do processo: pesquisa, ideação, prototipação, desenvolvimento e teste.
- Análise de casos de uso: como a IA está acelerando a criação de produtos digitais.

Unidade 2: Prototipação e Geração de UI com Inteligência Artificial

- Geração de wireframes e mockups a partir de descrições em linguagem natural (Text-to-UI).
- Ferramentas de prototipação visual assistida por IA.



- Mão na Massa: Utilizando o Firebase Studio para transformar designs do Figma em código front-end e analisar os resultados.
- Validação de usabilidade e fluxos de usuário com o auxílio de modelos de IA.

Unidade 3: Agentes de IA e CLI no Fluxo de Trabalho do Desenvolvedor Front-End

- O que são agentes de codificação e como eles operam no ambiente de desenvolvimento.
- Utilizando o Gemini CLI para scaffolding de projetos, geração de componentes e refatoração de código.
- Definindo um fluxo de trabalho com agentes de IA: da ideia ao componente funcional e testável.
- Boas práticas para engenharia de prompt focada em geração de código (HTML, CSS, JavaScript/TypeScript).

Unidade 4: Automação e Interação Inteligente com a UI

- Automação de testes End-to-End (E2E) com agentes autônomos que consomem o MCP da aplicação.
- Discussão Prática: Utilizando uma ferramenta como o mcpui.dev para visualizar e interagir com o contexto exposto pela aplicação via MCP, facilitando a depuração e o desenvolvimento de agentes.

Unidade 5: Integração de Lógica de IA no Cliente e no Servidor

- Consumindo APIs de IA diretamente do front-end: casos de uso e desafios de segurança.
- Utilizando Firebase AI Logic para criar, gerenciar e implantar backends customizados para aplicações front-end com lógica de IA embarcada.
- Desenvolvimento de features inteligentes: busca semântica, chatbots de suporte e personalização de conteúdo em tempo real.
- Mão na Massa: Construindo uma aplicação que utiliza lógica de IA hospedada no Firebase para processar inputs do usuário.

6 - Ferramentas de IA para DevOps

UNIDADE I - Fundamentos de IA Generativa para Infraestrutura

- 1.1. Arquitetura de Large Language Models (LLMs)
- 1.2. APIs de IA: OpenAI, Anthropic Claude, AWS Bedrock
- 1.3. Frameworks de agentes autônomos: LangChain, CrewAI, AutoGen
- 1.4. RAG (Retrieval-Augmented Generation) aplicado a documentação técnica
- 1.5. Padrões de prompting para tarefas de infraestrutura
- 1.6. Limitações, alucinações e estratégias de validação



UNIDADE II - IaC Copilot: Geração Inteligente de Infraestrutura

- 2.1. Tradução de linguagem natural para Infrastructure as Code
- 2.2. Geração automatizada de Terraform, Pulumi e Helm Charts
- 2.3. Validação automatizada: terraform plan, terraform validate
- 2.4. Policy-as-Code com OPA (Open Policy Agent) e Sentinel
- 2.5. Workflow PR-first com revisão assistida por IA
- 2.6. Detecção de drift de infraestrutura e sugestões de correção

Prática 1: Desenvolvimento de copilot para geração de módulos Terraform validados

UNIDADE III - Agentes para Kubernetes: Deploy e Operação

- 3.1. Geração e ajuste de manifests YAML via processamento de linguagem natural
- 3.2. Configuração inteligente de HorizontalPodAutoscaler (HPA) e VerticalPodAutoscaler (VPA)
- 3.3. Autoscaling baseado em workload e métricas customizadas
- 3.4. Gerenciamento automatizado de Ingress, Services e Network Policies
- 3.5. Estratégias de rollout/rollback guiadas por IA (canary, blue-green, progressive delivery)
- 3.6. Integração com GitOps: Argo CD e Flux

Prática 2: Agente para deployment end-to-end com validação de readiness

UNIDADE IV - Troubleshooting Assistido e Diagnóstico Inteligente

- 4.1. Framework ReAct (Reasoning + Acting) para troubleshooting estruturado
- 4.2. Diagnóstico automatizado de erros comuns em Kubernetes - CrashLoopBackOff, ImagePullBackOff, OOMKilled
- 4.3. Análise inteligente de logs distribuídos e stack traces
- 4.4. Correlação de traces (Jaeger, Zipkin) e métricas para root cause analysis
- 4.5. Detecção de latência, erros 5xx e degradação de performance
- 4.6. Sugestão automatizada de fixes e remediações

Prática 3 : Sistema de diagnóstico automático de problemas em pods

UNIDADE V - AIOps e Observabilidade Inteligente

- 5.1. Geração de queries PromQL e LogQL a partir de linguagem natural
- 5.2. Criação automatizada de dashboards Grafana contextualizados
- 5.3. Detecção de anomalias com Machine Learning (Prophet, Isolation Forest)
- 5.4. Correlação inteligente entre métricas, logs e traces
- 5.5. Alertas preditivos e redução de falsos positivos
- 5.6. Análise de tendências e forecasting de recursos

Prática 4: Dashboard inteligente com detecção de anomalias em tempo real

UNIDADE VI - ChatOps com Aprovação Humana e Governança

- 6.1. Arquitetura de bots conversacionais para Slack, Discord e Microsoft Teams
- 6.2. Implementação de comandos seguros: /deploy, /scale, /rollback, /investigar
- 6.3. Visualização de diffs e análise de impacto de mudanças
- 6.4. Workflows de aprovação com guardrails e circuit breakers
- 6.5. Integração com RBAC (Role-Based Access Control)
- 6.6. Auditoria e rastreabilidade de ações executadas

Prática 5: Bot de ChatOps com fluxo de aprovação para deploys

UNIDADE VII - Segurança e Compliance Assistidos por IA

- 7.1. Scan automatizado de IaC e containers (Snyk, Trivy, Checkov)
- 7.2. "AI fix" automático de vulnerabilidades detectadas
- 7.3. Detecção e prevenção de vazamento de secrets
- 7.4. Explicação de políticas OPA em linguagem natural
- 7.5. Remediação guiada de CVEs e vulnerabilidades críticas
- 7.6. Geração de relatórios de compliance (SOC2, ISO 27001, PCI-DSS)

UNIDADE VIII - CI/CD Copilot e Pipelines Inteligentes

- 8.1. Geração automatizada de pipelines (GitLab CI, GitHub Actions, Jenkins)
- 8.2. Gates inteligentes: testes automatizados, SAST, DAST, dependency scanning
- 8.3. Análise de impacto de mudanças em Pull Requests
- 8.4. Recomendações de estratégias de deployment (canary, blue-green)
- 8.5. Otimização de tempo de build e cache inteligente
- 8.6. Rollback automático baseado em métricas de qualidade

UNIDADE IX - FinOps e Otimização de Custos com IA

- 9.1. Estimativa de custos em Pull Requests com Infracost
- 9.2. Rightsizing de pods e nodes baseado em histórico de uso
- 9.3. Recomendações de instâncias spot e reservadas
- 9.4. Detecção de recursos ociosos e "zombie resources"
- 9.5. Forecasting de custos com Machine Learning
- 9.6. Otimização de armazenamento e networking

UNIDADE X - RAG de Runbooks e Post-mortem Automático

- 10.1. Arquitetura de RAG para documentação técnica interna
- 10.2. Indexação de playbooks, READMEs e wikis corporativos
- 10.3. Agente que consulta e executa runbooks automaticamente
- 10.4. Geração automática de timelines de incidentes



- 10.5. Extração de lições aprendidas (lessons learned)
- 10.6. Templates de post-mortem blameless automatizados

UNIDADE XI - Auto-Remediação Segura com Guardrails

- 11.1. Playbooks acionados automaticamente por alertas
- 11.2. Fluxo: alerta → verificação → mitigação → validação
- 11.3. Canary automático com rollback inteligente
- 11.4. Circuit breakers e rate limiting em ações automatizadas
- 11.5. Simulação (dry-run) antes de execução em produção
- 11.6. Human-in-the-loop para ações críticas e de alto impacto

UNIDADE XII - Projeto Integrador e Casos Práticos

- 12.1. Desenvolvimento de agente ou copilot completo
- 12.2. Integração de múltiplos componentes (IaC + K8s + Observability)
- 12.3. Implementação de guardrails e políticas de segurança
- 12.4. Testes de validação e simulação de cenários reais
- 12.5. Apresentação de projetos e discussão de resultados
- 12.6. Análise de viabilidade, ROI e melhores práticas

7 - Ferramentas de IA para Gestão de projetos

Unidade 1: Planejamento e Escopo com IA (Requirements Copilot)

- Aplicar ferramentas de IA para apoiar o levantamento e a definição de requisitos de software;
- Converter descrições em linguagem natural (NL) em épicos, histórias de usuário e requisitos de mudança;
- Realizar a decomposição de tarefas e definição de critérios de aceite com suporte de modelos generativos.

Unidade 2: Priorização Inteligente de Backlog

- Utilizar técnicas e frameworks de priorização (RICE, WSJF, MoSCoW) com suporte de IA;
- Simular impacto de funcionalidades e gerar sugestões automáticas de roadmap;
- Aplicar critérios de pontuação inteligente para refino de backlog.

Unidade 3: Cronograma, Capacidade e Alocação Assistidos

- Empregar IA para criação dinâmica de cronogramas e identificação de dependências entre tarefas;
- Analisar cenários “what-if” e realizar balanceamento de capacidade da equipe;
- Automatizar ajustes de cronograma com base em previsões de carga e restrições operacionais.

Unidade 4: Estimativas e Previsões

- Aplicar técnicas de previsão de prazos, custos e esforço baseadas em IA;
- Analisar dados históricos para melhoria da acurácia de estimativas;
- Empregar simulações probabilísticas (ex.: Monte Carlo) para análise de incertezas.

Unidade 5: Riscos e Mitigações com AIOps de Projeto

- Utilizar técnicas de AIOps para detecção automática de riscos de escopo, cronograma e recursos;
- Analisar padrões e anomalias em dados de projetos para prever falhas e gargalos;
- Gerar planos de mitigação automatizados com base em dados históricos e heurísticas.

Unidade 6: Reuniões Turbinadas (Notas, AIs, Follow-ups)

- Aplicar modelos de IA para transcrever e resumir reuniões de equipe;
- Identificar decisões, pendências e ações a partir de conversas em linguagem natural;
- Integrar soluções de IA a calendários e boards de projeto.

Unidade 7: Status Reports e Executive Summaries

- Automatizar a geração de relatórios de sprint, projeto e portfólio com IA;
- Adaptar comunicações para diferentes públicos (técnico, gestor, diretoria);
- Consolidar dados de desempenho e produtividade em relatórios em linguagem natural.

Unidade 8: Governança, Compliance e Qualidade

- Aplicar IA para verificação de conformidade e rastreabilidade em projetos;
- Criar checklists automatizados e trilhas de decisão para auditorias internas;
- Automatizar a geração e controle de documentação de qualidade;
- Implementar verificações automáticas em JavaScript para auditoria e controle de versões.



Unidade 9: Automação em Jira/Asana/Trello/Notion/Slack

- Criar automações inteligentes integradas a plataformas de gestão e colaboração;
- Desenvolver bots em JavaScript para criação, atualização e priorização de cards e tarefas;
- Utilizar linguagem natural (NL → workflow) para geração e aprovação de atividades;
- Aplicar regras de negócio automatizadas para lembretes, notificações e aprovações.

Unidade 10: Portfólio e OKRs com IA

- Aplicar IA para alinhamento de projetos com objetivos e resultados-chave (OKRs);
- Analisar métricas de outcome e priorizar iniciativas em portfólios corporativos;
- Avaliar impacto de projetos e propor ajustes baseados em dados.

8- Arquitetura de Sistemas com IA

Unidade 1: Fundamentos de Arquitetura AI-First

- Compreender as diferenças entre arquiteturas tradicionais e arquiteturas orientadas por Inteligência Artificial (AI-driven);
- Analisar os cinco principais padrões de design em sistemas inteligentes: *Prompting*, *Responsible AI*, *UX*, *AI-Ops* e *Optimization*;
- Aplicar um *decision framework* para identificar quando utilizar IA versus regras determinísticas tradicionais;
- Discutir trade-offs entre latência, precisão, custo e performance.

Unidade 2: Arquiteturas Single-Agent

- Compreender o conceito de agentes autônomos e seu papel em fluxos lineares de decisão;
- Analisar padrões de agente: *Reactive*, *Memory-Enhanced*, *Tool-Using*, *ReAct* e *Self-Reflection*;
- Identificar contextos ideais de aplicação (tarefas curtas, orçamentos limitados, baixa latência);
- Desenvolver protótipos de agentes simples em JavaScript integrando APIs de IA generativa;
- Simular interações entre o agente e usuários em ambientes de software reais.

Unidade 3: Arquiteturas Multi-Agent

- Entender o papel da colaboração entre agentes e sua importância em sistemas complexos;
- Explorar padrões de orquestração multiagente: *Sequential*, *Parallel*, *Supervisor*, *Hierarchical*, *Group Chat* e *Handoff*;



- Avaliar desafios distribuídos como falhas de nó, coordenação, controle de estado e sincronização;
- Implementar, em JavaScript, exemplos de comunicação entre múltiplos agentes utilizando filas de mensagens e chamadas assíncronas;
- Discutir casos de uso de multiagentes em ambientes corporativos (ex.: pipelines de análise, chatbots colaborativos, agentes especialistas).

Unidade 4: Padrões de Design AI-Específicos

- Aplicar padrões de *Retrieval-Augmented Generation (RAG)*: *Basic*, *Hybrid Search*, *Multi-Index* e *Agentic RAG*;
- Utilizar estratégias de roteamento inteligente: *Model Router* e *Intent-Based Routing* para otimização de custo e latência;
- Implementar mecanismos de *caching* e reutilização de respostas (Semantic Cache, Prompt Cache, Response Streaming);
- Incorporar mecanismos de *Human-in-the-Loop (HITL)* como *Approval Gates*, *Confidence Thresholds* e *Audit Trails*.

Unidade 5: Arquitetura Enterprise

- Projetar arquiteturas completas de IA em produção, com foco em modularidade, escalabilidade e segurança;
- Mapear o *stack* completo: API Gateway → Orquestração (Kubernetes) → Serviços Compartilhados → Observabilidade;
- Adotar princípios de design como *Loose Coupling*, *Clear Interfaces* e *Policy-Driven Control*;
- Empregar práticas de *observabilidade em IA*, incluindo rastreamento de prompts, logs e métricas de performance;
- Discutir modelos de implantação em ambientes híbridos (Kubernetes, Serverless e Edge);
- Implementação de exemplos de simulação de roteamento de modelos e controle de custos (*Model Tiering*).

9 - Processamento de Dados e Fine-Tuning de Modelos

Unidade 1: Quando fazer fine-tuning (decision framework)

- Projetar arquiteturas completas de IA em produção, com foco em modularidade, escalabilidade e segurança;
- Mapear o *stack* completo: API Gateway → Orquestração (Kubernetes) → Serviços Compartilhados → Observabilidade;
- Adotar princípios de design como *Loose Coupling*, *Clear Interfaces* e *Policy-Driven Control*;

Unidade 2: Preparação de Datasets para Fine-Tuning

- Identificar e coletar dados relevantes para o treinamento de modelos customizados;
- Aplicar técnicas básicas de limpeza, balanceamento e padronização de dados textuais;
- Estruturar datasets no formato JSONL, conforme especificações de APIs de fine-tuning;
- Automatizar a preparação de dados com scripts em JavaScript utilizando bibliotecas de manipulação de texto;
- Avaliar a importância de qualidade e diversidade dos dados no resultado final do modelo.

Unidade 3: Fine-tuning via OpenAI API (upload e train)

- Compreender o processo de fine-tuning em APIs comerciais (ex.: OpenAI, Gemini.);
- Executar o upload de datasets e acompanhar o treinamento do modelo passo a passo;
- Configurar hiperparâmetros básicos e monitorar o progresso do treinamento.
- Implementar scripts em JavaScript para automação do processo de upload e acompanhamento do fine-tuning;
- Discutir boas práticas de versionamento e documentação de modelos fine-tunados.

Unidade 4: LoRA e PEFT (conceitos práticos)

- Entender os princípios de *Low-Rank Adaptation (LoRA)* e *Parameter-Efficient Fine-Tuning (PEFT)*;
- Aplicar estratégias de fine-tuning leve para reduzir custos e acelerar o treinamento;
- Avaliar trade-offs entre desempenho e eficiência de parametrização;
- Implementar demonstrações práticas em JavaScript utilizando APIs que suportam camadas adaptativas;
- Comparar o comportamento de modelos full fine-tuned vs LoRA em tarefas específicas.

Unidade 5: Avaliar modelos fine-tunados

- Aplicar métricas de avaliação qualitativas e quantitativas para modelos ajustados;
- Medir precisão, consistência e adequação das respostas em diferentes cenários;
- Utilizar testes A/B para comparar modelos customizados e genéricos;
- Implementar scripts em JavaScript para automação de testes de desempenho e logging de resultados;
- Discutir riscos de *overfitting* e perda de generalização em modelos ajustados.

Unidade 6: Projeto Final: Modelo Customizado para Domínio Específico

- Planejar e desenvolver um modelo fine-tunado voltado a um domínio específico (ex.: atendimento, suporte técnico, jurídico, financeiro);



- Aplicar os conceitos aprendidos de preparação de dados, ajuste e avaliação;
- Integrar o modelo customizado em um fluxo prático de uso, como um chatbot ou agente automatizado;
- Documentar e apresentar os resultados, justificando as escolhas de arquitetura e abordagem;
- Implementar em JavaScript um protótipo funcional que utilize o modelo treinado via API.

10 - Segurança e Governança em IA

- O que é Governança em IA
- Interpretabilidade e Explicabilidade em Modelos de IA
- Vieses e Responsabilidade em Modelos de IA
- Gerenciamento de Riscos em Inteligência Artificial - Aspectos Humanos e Éticos
- Gerenciamento de Riscos em Inteligência Artificial - Segurança e Dados
- Gerenciamento de Riscos em Inteligência Artificial - Aspectos Legais e Regulatórios
- Custos Financeiros em Inteligência Artificial

11 - Projeto Integrador – Capstone Project

Unidade 1: Ideação e Arquitetura do Micro-SaaS

- Definição do problema e do escopo do Micro-SaaS.
- Análise de viabilidade: identificação do público-alvo e construção da proposta de valor.
- Desenho da arquitetura da solução, contemplando front-end, back-end e os componentes de IA (RAG, Agentes, Orquestração).
- Planejamento do projeto: definição de milestones, backlog inicial e tecnologias (stack).

Unidade 2: Fundamentos da Solução - RAG e Agentes

- Construção do core da inteligência: implementação de um sistema de Retrieval-Augmented Generation (RAG).
- Seleção e configuração de Vector Databases
- Desenvolvimento do agente principal que utilizará o RAG para responder a queries complexas.
- Entrega 1: Protótipo funcional do sistema RAG com uma interface de linha de comando (CLI) para validação da lógica.

Unidade 3: Orquestração, Back-end e a Interface Inteligente

- Desenvolvimento da integração com o back-end: criação de APIs
- Implementando a Ponte IA-UI: Habilitando o Model-Context Protocol (MCP) na aplicação para expor as capacidades do serviço de forma padronizada a agentes.
- Orquestração de múltiplos agentes ou LLMs usando o ADK (Agent Development Kit).



- Entrega 2: API do back-end documentada e funcional, e a aplicação front-end com o MCP habilitado, pronta para interação programática.

Unidade 4: Desenvolvimento do Front-End, Interação e Implantação

- Desenvolvimento da interface do usuário (UI) do Micro-SaaS em Angular, já estruturada com o MCP.
- Integração do front-end com as APIs do back-end para consumir os serviços de IA.
- Validação do MCP: Utilização de ferramentas para validar que os agentes conseguem interpretar e interagir corretamente com o front-end através do protocolo.
- Configuração do pipeline de CI/CD e implantação da solução.
- Entrega 3: Solução completa implantada, com a UI principal funcional e a capacidade de demonstrar a interação de um agente com o front-end via MCP.

Unidade 5: Apresentação Final e Defesa Técnica

- Estruturação da apresentação do projeto: storytelling, demonstração ao vivo e resultados alcançados.
- Preparação para a defesa técnica: capacidade de justificar as decisões de arquitetura, tecnologia e implementação.
- Finalização da documentação do projeto: README claro, diagrama de arquitetura e instruções de uso.
- Avaliação final com base na complexidade, funcionalidade, qualidade do código, apresentação e defesa do projeto.

12 - Carreira e Entrevistas para Engenheiros de AI Aplicada

- **Otimização de LinkedIn para Recrutadores** - Palavras-chave de IA, SSI alto, headline impactante, seção "Sobre" com projetos
- **Networking Estratégico** - Conectar com recrutadores e tech leads, participar de comunidades de IA, engajamento consistente
- **Marca Pessoal e Posicionamento** - Compartilhar aprendizados, artigos técnicos, projetos, posicionar-se como especialista em IA aplicada
- **Portfólio GitHub Profissional** - Projetos deployados, READMEs detalhados, código limpo, demonstrações ao vivo acessíveis
- **Expectativas por Nível (Junior → Principal)** - O que cada nível exige: escopo, autonomia, impacto, liderança técnica
- **Preparação para Entrevistas de RH** - Metodologia STAR, storytelling efetivo, alinhamento cultural, responder pontos fracos estrategicamente
- **Expectativas Salariais e Proposta de Valor** - Pesquisa de mercado, como justificar seu valor, quando mencionar números
- **Live Coding e Problem Solving** - Prática de coding interviews, pensar em voz alta, testar casos edge, otimização



- **System Design para Sistemas de IA** - Arquitetar sistemas completos, justificar trade-offs, escalabilidade, custos, resiliência
- **Explicar Decisões Técnicas** - Comunicar escolhas arquiteturais, por que escolheu determinado padrão, alternativas consideradas
- **Perguntas Técnicas Específicas de IA** - RAG vs fine-tuning, quando usar agents, otimização de custos, handling de failures
- **Negociação Salarial Efetiva** - Timing correto, âncoras salariais, múltiplas ofertas como leverage, pacote completo além do salário
- **Pedido de Aumento/Promoção** - Documentar impacto, timing estratégico, construir caso com métricas, apresentação profissional
- **Técnicas de Diferenciação** - Demonstrar projetos ao vivo, case studies de resultados, contributions open source, presença técnica